МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ
Ордена Трудового Красного Знамени
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Московский технический университет связи и информатики»

Кафедра «Программная инженерия»

ОТЧЕТ
по лабораторной работе №5

по дисциплине «Основы DevOps»
по теме «Методология CI/CD»

Выполнил: студентка группы БПИ2401
Еланская Екатерина Дмитриевна
Проверил: Тимчук А.В.

Москва, 2026

Цель работы

— познакомиться с методологией CI/CD;

— научиться работать с инструментом GitHub Actions.

Задачи:

1) изучить методические материалы к лабораторной работе;

2) написать тесты для приложения на Python;

3) написать пайплайн сборки, тестирования и загрузки образа приложения на Docker Hub.

Выполнение задания

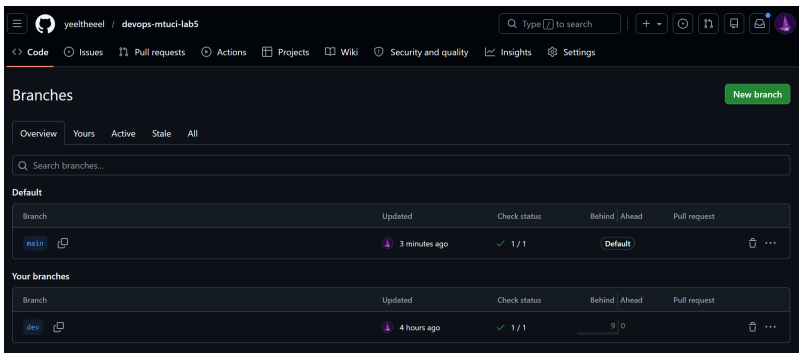

Рисунок 1 — Новый репозиторий с веткой dev

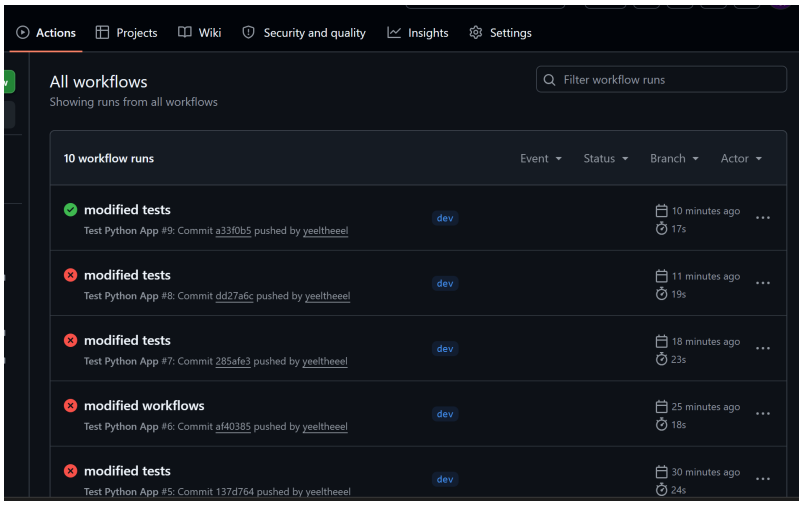

Рисунок 2 — Работа пайплайна в ветке dev

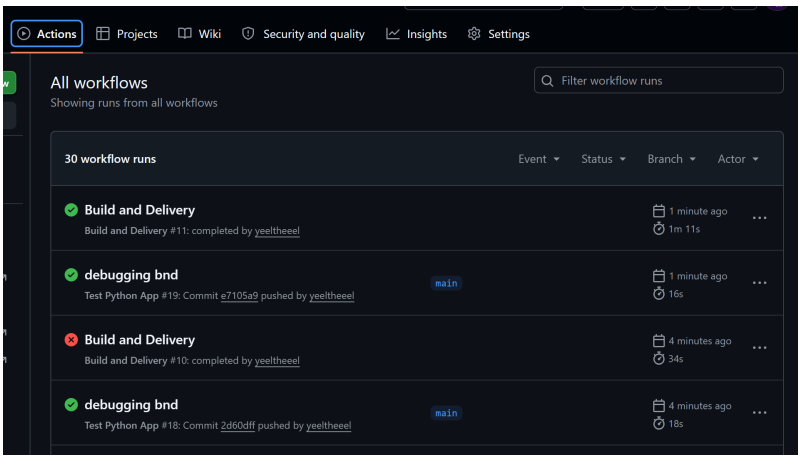

Рисунок 3 — Работа пайплайнов в ветке main

Рисунок 4 — Загруженный образ в Docker Hub

Рисунок 5 — Запущенный образ

Исходный код программы

```
from fastapi.testclient import TestClient
from src.main import app

client = TestClient(app)
users = [
    {
        'id': 1,
        'name': 'Ivan Ivanov',
        'email': 'i.i.ivanov@mail.com',
    },
    {
        'id': 2,
        'name': 'Petr Petrov',
        'email': 'p.p.petrov@mail.com',
    }
]

def test_get_existed_user():
    response = client.get("/api/v1/user", params={'email': users[0]['email']})
    assert response.status_code == 200
    assert response.json() == users[0]

def test_get_unexisted_user():
    response = client.get("/api/v1/user", params={'email': 'none@mail.com'})
    assert response.status_code == 404

def test_create_user_with_valid_email():
```

```python
    new_user = {
        'name': 'Mark Markovich',
        'email': 'm.m.markovich@mail.com'
    }
    response = client.post("/api/v1/user", json=new_user)
    assert response.status_code == 201

def test_create_user_with_invalid_email():
    new_user = {
        'name': 'Fakevan Fakevanov',
        'email': 'i.i.ivanov@mail.com'
    }
    response = client.post("/api/v1/user", json=new_user)
    assert response.status_code == 409

def test_delete_user():
    new_user = {
        'name': 'test user',
        'email': 'test@mail.com'
    }
    response = client.post("/api/v1/user", json=new_user)
    response = client.get("/api/v1/user", params={'email': 'test@mail.com'})
    assert response.status_code == 200
```

Листинг 1 – Дополненные методы тестирования

```yaml
name: Test Python App
on:
  push:
    branches: ['main', 'dev']
jobs:
  ci:
    runs-on: ubuntu-latest
    steps:
    - name: Get repository
      uses: actions/checkout@v5
    - name: Setup python
      uses: actions/setup-python@v6
      with:
        python-version: '3.12'
    - name: Install dependencies
      run: |
        python -m pip install --upgrade pip
        pip install pytest httpx fastapi uvicorn pydantic_settings
    - name: Run tests
      run: python -m pytest tests/
```

Листинг 2 – Пайплайн тестирования

```yaml
name: Build and Delivery
on:
  workflow_run:
    workflows: ["Test Python App"]
    types: [completed]
    branches: ['main']
jobs:
  cd:
    runs-on: ubuntu-latest
    if: ${{ github.event.workflow_run.conclusion == 'success' }}
    steps:
```

```yaml
    - name: Login to Docker Hub
      uses: docker/login-action@v3
      with:
        registry: ghcr.io
        username: ${{ github.actor }}
        password: ${{ secrets.GITHUB_TOKEN }}
    - name: Push image
      uses: docker/build-push-action@v6
      with:
        push: true
        tags: ghcr.io/${{ vars.DOCKERHUB_USERNAME }}/my-app:latest
    permissions:
      contents: read
      packages: write
```

Листинг 3 – Пайплайн сборки и загрузки образа приложения

Заключение

В ходе выполнения лабораторной работы были решены следующие задачи:

— проведено ознакомление с методологией CI/CD;

— получены знания о том, как работать с инструментом GitHub Actions.

Контрольные вопросы

1) *Как CI/CD помогает в разработке?* Помогает избежать ошибок и сбоев в коде, поддерживая непрерывный цикл разработки и обновления программного обеспечения. Поскольку CI/CD автоматизирует ручное вмешательство человека, время простоя сводится к минимуму, а релизы кода происходят быстрее, снижается человеческий фактор.

2) *К какому этапу CI/CD относится сборка десктоп приложения под операционные системы Windows, MacOS и Linux для его дальнейшего тестирования?* ответ.

3) *Что такое Continuous Delivery и Deploy?* Continuous Delivery (непрерывная доставка) автоматизирует выпуск проверенного кода в репозиторий после сборки и тестирования приложения на этапе CI. Deploy (непрерывное развертывание) – заключительный этап конвейера CI/CD, является расширением непрерывной доставки и может означать автоматизацию выпуска изменений из репозитория в производство.